

Production memory

(memory bank)

Introduction

The memory gap

After part 2, you deployed agents with session persistence. But there's one gap:

```
Python
# Session 1 (Monday):
User: "I prefer email notifications over SMS"
Agent: "Noted!"

# Session 2 (Friday - different conversation):
User: "How should I be notified?"
Agent: "I don't remember" ❌ # Session state is conversation-scoped
```

The gap: Session state only persists within a conversation.
Memory bank enables learning across all conversations.

Session state vs memory bank

Quick comparison

Feature	Session state (course 4)	Memory bank
Scope	Current conversation	All conversations
Persistence	End of session	Indefinitely
Use case	Conversation flow	Long-term learning
Access	Direct (<code>{variable}</code>)	Via tool (semantic search)

When to use each

Session state (course 4):

- Current topic: `temp:current_topic`
- Request count: `request_count`
- User preferences: `user:tier`

Memory bank (course 9):

- “User prefers email notifications” (learned last week)
- “Had issue with billing before” (from last month)
- “Interested in Enterprise API” (from multiple conversations)

They work together

None

Session State: "What's happening THIS conversation?"

Memory Bank: "What have I learned ACROSS ALL conversations?"

How memory bank works

The lifecycle (conceptual)

None

1. User interacts with agent
2. Conversation saved to Memory Bank (automatic via callback)
3. LLM extracts meaningful information (preferences, interests, etc.)
4. Later conversation: Agent uses tool to search past memories
5. Agent responds with historical context

Key components

VertexAiMemoryBankService:

- Managed service (no database setup)
- LLM-powered extraction (intelligent, not raw storage)
- Semantic search (understands intent)

PreloadMemoryTool:

- Automatically retrieves relevant memories
- Agent uses memories to inform responses

Auto-save callback:

Python

After each turn, save to Memory Bank

```
async def auto_save_session_to_memory_callback(callback_context):
```

```
    await callback_context._invocation_context.memory_service.add_session_to_memory(
        callback_context._invocation_context.session)
```